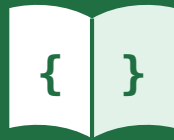

Modül 07 — Spring Data MongoDB

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Belge Modeli	2
2.2 Gömme mi, Referans mı?	2
2.3 Spring Boot 4'te MongoDB Ayarları	3
3. Adım Adım Örnekler	3
3.1 Belge Modelleme	3
3.2 Repository Sorguları	4
3.3 MongoTemplate: Dinamik Sorgular ve Atomik Güncellemeler	5
3.4 Aggregation Pipeline	6
3.5 Index'ler	6
3.6 Çok Belgeli Transaction'lar	7
3.7 Testcontainers ile Test	8
4. Sık Yapılan Hatalar ve En İyi Pratikler	9
5. Özet	9
6. İleri Okuma	10

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Veriyi belge (document) olarak modellemek; gömme (embed) ile referans arasında seçim yapmak
- MongoRepository ile türetilmiş ve JSON sorgular yazmak
- MongoTemplate ile dinamik sorgular ve atomik güncellemeler yapmak
- Aggregation pipeline ile raporları veritabanında hesaplamak
- Tekil ve metin (text) index'leri tanımlamak
- Replica set üzerinde çok belgeli transaction kullanmak
- MongoDB'yi Testcontainers ile test etmek

Ön koşullar: Modül 05-06 (Spring Data, transaction'lar, Testcontainers) · **Tahmini süre:** 5 saat · **Docker gerekir**

2. Kavramlar

2.1 Belge Modeli

MongoDB, veriyi tablolarda değil, JSON benzeri **belgelerde** (BSON) saklar. Bir belge iç içe nesneler ve diziler içerebilir. Aynı koleksiyondaki belgelerin alanları farklı olabilir.

İlişkisel (PostgreSQL)	MongoDB
Tablo	Koleksiyon
Satır	Belge
Sütun	Alan
JOIN	Gömme veya \$lookup / referans
Şema (DDL)	Esnek. Kuralları uygulama ve index'ler korur

2.2 Gömme mi, Referans mı?

Gömün (embed)	Referans verin
Birlikte okunur (kitap + yorumları)	Bağımsız okunur veya değişir (yayınevi)
Sayısı sınırlıdır	Sınırsız büyüyebilir
Tek başına anlamı yoktur	Birçok belgede paylaşılır

i Not

Bir belge en fazla 16 MB olabilir. Sınırsız büyüyen listeleri (ör. bir kitabın tüm satışları) gömmeyin. Ayrı bir koleksiyona koyun.

2.3 Spring Boot 4'te MongoDB Ayarları

Bağlantı ayarları `spring.mongodb.*` altındadır. Eski `spring.data.mongodb.uri` gibi anahtarlar kullanımdan kalkmıştır. Spring Data'ya özgü ayarlar ise `spring.data.mongodb.*` altında kalır.

3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın. MongoDB, bir replica set olarak otomatik başlar:

```
./mvnw -pl modules/07-data-mongodb/lesson -am spring-boot:run
```

MongoDB'de migration yoktur. Tur, koleksiyon boşsa örnek veriyi kendisi ekler. Ayarlar:

```
mongodb:
  database: bookstore                                # Boot 4: connection settings live under
  ↪ spring.mongodb.*
data:
  mongodb:
    auto-index-creation: true                        # create @Indexed / @TextIndexed indexes at
    ↪ startup
    representation:
      big-decimal: decimal128                        # exact decimals (the current default) – stated
    ↪ explicitly
```

3.1 Belge Modelleme

Amaç: Bir kitabı yorumları gömülü, yayınevi referanslı ve serbest özellikli tek bir belge olarak saklamak.

```
@Document("books")
public record Book(
    @Id @Nullable String id,
    @Indexed(unique = true) String isbn,           // Lesson 3.5: a unique index
    @TextIndexed String title,                   // Lesson 3.5: part of the
    ↪ full-text index
```

```

List<String> authors,           // arrays are first-class in
    ↪ MongoDB
BigDecimal price,             // stored as Decimal128 (see
    ↪ application.yaml)
int stock,
List<String> categories,
Map<String, String> attributes, // different books, different
    ↪ attributes – no schema change
@DocumentReference @Nullable Publisher publisher, // stores only the
    ↪ publisher's id
List<Review> reviews) {       // embedded: stored inside
    ↪ this document

```

Beklenen çıktı (veritabanındaki ham belge):

```

== 3.1 A document
{"isbn": "9780134685991", "title": "Effective Java", "authors": ["Joshua Bloch"],
 "price": {"$numberDecimal": "89.90"}, "stock": 5, "categories": ["java",
    ↪ "best-practices"],
 "attributes": {"language": "en", "pages": "412"}, "publisher": {"$oid": "6ab4..."},
 "reviews": [{"author": "ayse", "stars": 5, "text": "Harika"}, ...]}

```

Yorumlar belgenin içindedir. Yayınevinden ise yalnızca id saklanır ve okurken Spring Data onu çözer.

Testi: catalog/BookRepositoryTest.reviewsAreEmbeddedAndThePublisherIsReferenced

i Not

BigDecimal değerleri kesin ondalık tipi Decimal128 olarak saklanır. Bu, güncel Spring Data MongoDB'nin varsayılanıdır. Biz yine de representation.big-decimal: decimal128 ile açıkça yazıyoruz. Eski sürümler BigDecimal'ı **metin** olarak saklıyordu ve o durumda \$sum/\$avg çalışmaz, sıralama alfabetik olur ("100" < "9").

3.2 Repository Sorguları

Amaç: JPA'daki repository modelini MongoDB ile kullanmak.

```

public interface BookRepository extends MongoRepository<Book, String> {

    Optional<Book> findByIsbn(String isbn);

    List<Book> findByAuthorsContaining(String author);           // matches one
    ↪ element of the array

```

```

List<Book> findByPriceLessThanOrderByPrice(BigDecimal max);

@Query("{ 'attributes.language': ?0 }") // a MongoDB JSON
↪ query
List<Book> findByLanguage(String language);
}

```

Beklenen çıktı:

```

== 3.2 Repository queries
by author:    [Effective Java, Java Puzzlers]
by language:  [Kürk Mantolu Madonna]
publisher:    Publisher[id=6ab4..., name=Addison-Wesley, country=US]

```

3.3 MongoTemplate: Dinamik Sorgular ve Atomik Güncellemeler

Amaç: Değişken kriterlerle sorgu kurmak ve bir belgeyi okumadan, tek adımda güncellemek.

```

public List<Book> search(@Nullable String category, @Nullable BigDecimal maxPrice) {
    List<Criteria> parts = new ArrayList<>();
    if (category != null) {
        parts.add(where("categories").is(category)); // "is" on an array field =
↪ "contains"
    }
    if (maxPrice != null) {
        parts.add(where("price").lte(maxPrice));
    }
    Query query = parts.isEmpty() ? new Query() : new Query(new
↪ Criteria().andOperator(parts));
    return mongo.find(query.with(Sort.by("price")), Book.class);
}

```

Stok azaltma, “yeterli stok var mı?” kontrolünü ve değişikliği **tek bir atomik işlemde** yapar. İki eşzamanlı istek stoğu eksiye düşüremez:

```

public boolean takeFromStock(String isbn, int quantity) {
    var onlyIfEnough = new Query(where("isbn").is(isbn).and("stock").gte(quantity));
    var result = mongo.updateFirst(onlyIfEnough, new Update().inc("stock", -quantity),
↪ Book.class);
    return result.getModifiedCount() == 1; // check and change in ONE atomic
↪ operation
}

```

```
public void addReview(String isbn, Review review) {
    mongo.updateFirst(new Query(where("isbn").is(isbn)), new Update().push("reviews",
    ↪ review), Book.class);
}
```

Beklenen çıktı:

```
== 3.3 MongoTemplate
java ≤ 90:    [Java Puzzlers, Effective Java]
take 1 copy: true, stock now 9
```

3.4 Aggregation Pipeline

Amaç: Kategori başına kitap sayısını ve ortalama fiyatı veritabanında hesaplamak.

Pipeline, belgeleri sırayla aşamalardan geçirir: \$unwind bir diziyi satırlara açar, \$group gruplar, \$project şekillendirir, \$sort sıralar:

```
public List<CategoryStats> statsPerCategory() {
    var pipeline = newAggregation(
        unwind("categories"), // one row per
        ↪ (book, category)
        group("categories").count().as("count").avg("price").as("averagePrice"),
        project("count")
            .and("_id").as("category")

        ↪ .and(ArithmeticOperators.Round.roundValueOf("averagePrice").place(2)).as("averagePrice")
        sort(Sort.by(Sort.Order.desc("count"), Sort.Order.asc("category"))));
    return mongo.aggregate(pipeline, Book.class, CategoryStats.class).getMappedResults();
}
```

Beklenen çıktı:

```
== 3.4 Aggregation
CategoryStats[category=java, count=3, averagePrice=79.97]
CategoryStats[category=best-practices, count=1, averagePrice=89.90]
CategoryStats[category=roman, count=1, averagePrice=45.00]
CategoryStats[category=spring, count=1, averagePrice=95.00]
```

3.5 Index'ler

Amaç: Sorguları hızlandırmak ve kuralları (tekil ISBN) veritabanında korumak.

@Indexed(unique = true) ve @TextIndexed açıklamaları (bölüm 3.1), auto-index-creation: true ile başlangıçta index'e dönüşür. Aynı ISBN'le ikinci bir kitap DuplicateKeyException fırlatır. Metin index'i, kelime köklerini de tanıyan tam metin aramayı mümkün kılar:

```
public List<Book> fullText(String words) {
    return
        ↪ mongo.find(TextQuery.queryText(TextCriteria.forDefaultLanguage()).matchingAny(words)),
        ↪ Book.class);
}
```

Beklenen çıktı:

```
== 3.5 Text index
"puzzlers" → [Java Puzzlers]
```

Testi: catalog/CatalogOperationsTest

★ İpucu

Üretimde index'leri uygulama başlangıcına bırakmak yerine (büyük koleksiyonlarda uzun sürebilir), mongo.indexOps(...) ile bilinçli olarak veya bir migration aracıyla yönetin.

3.6 Çok Belgeli Transaction'lar

Amaç: Sipariş kaydı ve stok düşmeyi, iki farklı koleksiyonda, hepsi-ya-hiçbiri olarak yapmak.

Spring Boot, MongoDB için bir transaction manager kaydetmez. Onu biz tanımlarız:

```
@Configuration(proxyBeanMethods = false)
public class MongoTransactionConfiguration {

    @Bean
    MongoTransactionManager transactionManager(MongoDatabaseFactory databaseFactory) {
        return new MongoTransactionManager(databaseFactory);
    }
}
```

```
@Service
public class OrderService {

    private final MongoTemplate mongo;
```

```

public OrderService(MongoTemplate mongo) {
    this.mongo = mongo;
}

@Transactional // uses the
    ↳ MongoTransactionManager bean
public void place(String customer, String isbn, int quantity) {
    mongo.insert(new Order(null, customer, isbn, quantity, Instant.now())); // 1st
    ↳ document ...

    Inventory inventory = mongo.findById(isbn, Inventory.class);
    if (inventory == null || inventory.quantity() < quantity) {
        throw new IllegalStateException("Not enough stock for " + isbn); // ... is
        ↳ rolled back here
    }
    mongo.save(new Inventory(isbn, inventory.quantity() - quantity)); // 2nd
    ↳ document
    }
}

```

Beklenen çıktı:

```
== 3.6 Transaction
```

```
rolled back: Not enough stock for 9781617297571 – orders stored: 0
```

Testi: orders/OrderServiceTest. Transaction manager bean'i kaldırıldığında bu test başarısız olur, çünkü sipariş belgesi geride kalır.

! Önemli

MongoDB transaction'ları yalnızca **replica set** (veya sharded cluster) üzerinde çalışır. compose.yaml, tek düğümlü bir replica set (rs0) başlatır. Testlerde bunu withReplicaSet() sağlar (bölüm 3.7).

3.7 Testcontainers ile Test

```

@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final MongoDBContainer MONGO = new MongoDBContainer("mongo:8.0.32") // same
    ↳ image as compose.yaml
      .withReplicaSet(); // Testcontainers 2: opt-in;
    ↳ needed for transactions
}

```

```
@Bean
@ServiceConnection
MongoDBContainer mongoContainer() {           // not "mongo": Boot already has a
↪ MongoClient bean with that name
    return MONGO;
}
```

@DataMongoTest, yalnızca MongoDB altyapısını ve repository'leri yükler. JPA'nın aksine her test sonunda **geri alma yoktur**. Bu yüzden testler @BeforeEach ile kendi verisini siler ve yeniden ekler.

! Uyarı

Container'ı döndüren @Bean metoduna mongo adını vermeyin. Boot'un MongoClient bean'i zaten bu adı taşır ve bean çakışması olur.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

"MongoDB şemasızdır" diye veri kalitesinden vazgeçmeyin. Tekil index'ler, uygulama doğrulaması ve gerekirse MongoDB'nin JSON Schema doğrulaması ile kuralları koruyun.

- **Yapın:** Belgeleri **nasıl okunduklarına** göre modelleyin. Birlikte okunanı gömün.
- **Yapmayın:** İlişkisel tabloları birebir koleksiyonlara çevirip her yerde referans kullanmayın. Her okuma birden çok sorguya dönüşür.
- **Yapın:** "Oku, değiştir, yaz" yerine \$inc, \$push, \$set gibi atomik güncellemeler kullanın.
- **Yapmayın:** Fiyat gibi değerleri double veya metin olarak saklamayın. Decimal128 kullanın.
- **Yapın:** Sık sorgulanan alanlara index koyun ve explain() ile sorgunun index kullandığını doğrulayın.

5. Özet

- MongoDB veriyi esnek belgelerde saklar. Gömme ve referans, okuma biçimine göre seçilir.
- MongoRepository, türetilmiş ve JSON sorgular sunar. MongoTemplate ise dinamik sorgular ve atomik güncellemeler için kullanılır.

- Aggregation pipeline, raporları veritabanında hesaplar.
- `@Indexed(unique = true)` ve `@TextIndexed` index'leri tanımlar.
- Çok belgeli transaction'lar replica set ve bir `MongoTransactionManager` bean'i ister.
- Boot 4'te bağlantı ayarları `spring.mongodb.*` altındadır. `BigDecimal` için `decimal128` kullanın.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Data MongoDB Reference](#)
- [Template Query Operations](#) · [Aggregation Framework](#)
- [Document References](#) · [Index Management](#)
- [Sessions & Transactions](#)
- [Spring Boot — NoSQL \(MongoDB\)](#)
- [MongoDB — Data Modeling](#)